LAPORAN PRAKTIKUM
MODUL 5: ANALISIS KOMPLEKSITAS ALGORITMA (BIG O, TIME & SPACE COMPLEXITY

Disusun oleh :

Nama Mahasiswa : Zahwa Aqiilah

Nim Mahasiswa : 2025573010066

Semester/Kelas : 02/TI-1D

Jurusan/Prodi : TIK/TI

Dosen Pengajar : Muhammad Reza Zulman, S.S.T,M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

TAHUN 2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum ini dengan baik. Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Struktur Data dan Algoritma, khususnya pada pembahasan mengenai Analisis Kompleksitas Algoritma menggunakan JavaScript dan Node.js.

Dalam penyusunan laporan ini, penulis mempelajari berbagai konsep penting seperti Big O Notation, Time Complexity, dan Space Complexity. Selain itu, praktikum ini juga memberikan pemahaman mengenai bagaimana cara menganalisis efisiensi suatu algoritma serta membandingkan performa antara algoritma yang berbeda melalui pengujian secara langsung. Dengan adanya praktikum ini, penulis menjadi lebih memahami pentingnya menulis kode yang tidak hanya benar, tetapi juga efisien dan optimal dalam penggunaan waktu maupun memori.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi materi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun guna perbaikan di masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca yang ingin memahami lebih dalam mengenai analisis kompleksitas algoritma.

Buketrata, 23 April 2026

Zahwa Aqiilah
NIM. 2025573010066

DAFTAR ISI

KATA PENGANTAR ..... **ii**
DAFTAR ISI ..... **iii**

BAB I PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 1
1.4 Rumusan Masalah 2

BAB II DASAR TEORI **3**
2.1 Pengertian Kompleksitas Algoritma 3
2.2 Big O Natation 3
2.3 Time Complexity 3
2.4 Space Complexity 3

BAB III PEMBAHASAN **4**
3.1 Alat dan Bahan 4
3.2 Langkah-Langkah 4
3.3 Hasil Praktikum 6
3.4 Submit ke GitHub 7

BAB IV PENUTUP **8**
4.1 Kesimpulan 8
4.2 Saran 8

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut pembuatan program yang tidak hanya benar secara logika, tetapi juga efisien dalam penggunaan waktu dan memori. Dalam pengolahan data berukuran kecil, perbedaan performa algoritma mungkin tidak terlalu terasa. Namun, ketika data yang diproses semakin besar seperti pada aplikasi modern (misalnya sistem pencarian, media sosial, atau e-commerce), efisiensi algoritma menjadi sangat penting karena dapat mempengaruhi kecepatan sistem dan pengalaman pengguna.

Analisis kompleksitas algoritma hadir sebagai solusi untuk mengukur dan membandingkan tingkat efisiensi suatu algoritma. Melalui konsep Big O Notation, kita dapat mengetahui bagaimana pertumbuhan waktu eksekusi (time complexity) dan penggunaan memori (space complexity) seiring bertambahnya ukuran input. Dengan memahami hal ini, seorang programmer dapat memilih atau merancang algoritma yang lebih optimal sesuai kebutuhan.

Selain itu, pemahaman tentang kompleksitas algoritma juga membantu dalam mengenali pola kode yang kurang efisien, seperti penggunaan nested loop yang berlebihan, serta mendorong penggunaan struktur data yang lebih tepat seperti Set atau Map. Hal ini penting agar program dapat berjalan lebih cepat dan hemat sumber daya.

Oleh karena itu, praktikum ini dilakukan untuk memberikan pemahaman dasar mengenai analisis kompleksitas algoritma, sehingga mahasiswa mampu mengevaluasi dan meningkatkan kualitas kode yang dibuat, baik dari segi performa maupun penggunaan memori.

1.2 Tujuan Praktikum

Adapun tujuann praktikum ini Adalah sebagai berikut :

1. Untuk memahami konsep dasar analisis kompleksitas algoritma.
2. Menjelaskan perbedaan antara time complexity dan space complexity.
3. Mampu membaca dan menuliskan notasi Big O.
4. Mampu menganalisis kompleksitas dari suatu algoritma.
5. Mampu mengoptimalkan kode agar lebih efisien.

1.3 Manfaat Praktikum

1. Membantu memhami pentingnya efisiensi algoritma dalam pembuatan program
2. Meningkatan kemampuan dalam menganalisis kinerja suatu algoritma
3. Melatih kemampuan dalam mengoprimalkan kode agar lebih cepat dan efisien.
4. Menjadi dasar dalam mempelajari struktur data dan algoritma yang lebih lanjut

1.4 Rumusan Masalah

1. Bagaimana cara memahami dan menjelaskan konsep dasar analisis kompleksitas algoritma, khususnya Big O Notation?

2. Bagaimana cara menganalisis serta membandingkan efisiensi algoritma berdasarkan time complexity dan space complexity?

BAB II
DASAR TEORI

2.1 Pengertian Kompleksitas Algoritma

Kompleksitas algoritma adalah cara untuk mengukur efisiensi suatu algoritma dalam menyelesaikan masalah. Efisiensi ini dilihat dari dua aspek utama, yaitu waktu eksekusi (time complexity) dan penggunaan memori (space complexity). Analisis ini penting karena program yang berjalan cepat pada data kecil belum tentu efisien saat digunakan pada data yang besar.

2.2 Big O Natation

Big O Notation digunakan untuk menggambarkan seberapa cepat pertumbuhan waktu atau memori suatu algoritma terhadap ukuran input. Big O tidak mengukur waktu secara langsung (detik), tetapi melihat pola pertumbuhannya.

Beberapa jenis Big O:

- **O(1)** → Konstan (tidak dipengaruhi jumlah data)
- **O(log n)** → Logaritmik (data dibagi dua setiap langkah)
- **O(n)** → Linear (sebanding dengan jumlah data)
- **O(n log n)** → Digunakan pada sorting cepat
- **O(n^2)** → Kuadratik (nested loop, lambat)

2.3 Time Complexity

Time complexity adalah ukuran seberapa lama waktu yang dibutuhkan algoritma untuk berjalan. Semakin kecil kompleksitasnya, semakin cepat algoritma tersebut.

Contoh:

- Loop tunggal → O(n)
- Nested loop → O(n^2)
- Operasi langsung → O(1)

2.4 Space Complexity

Space complexity adalah jumlah memori tambahan yang digunakan algoritma saat dijalankan.

Contoh:

- Menggunakan variabel biasa → O(1)
- Membuat array baru → O(n)
- Rekursi → O(n) karena call stack

BAB III
PEMBAHASAAN

3.1 Alat dan Bahan

Alat:

1. Laptop
2. Keyboard
3. Internet

Bahan:

1. Visual Studio Code
2. Git Bash
3. Node.jd

3.2 Langkah-Langkah

1. Membuka Git Bash
2. Lalu ketik cd/c
3. Ketik cd/”c/
4. Nama Repository
5. Mkdir praktikum-5
6. Cd praktikum-5
7. Npm init -y

```
LOQ@LAPTOP-337SK54S MINGW64 ~
$ cd /d

LOQ@LAPTOP-337SK54S MINGW64 /d
$ cd "/d/Praktikum Struktur Data dan Algoritma"

LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma
$ cd 2025573010066_strukturdataalgoritma

LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritma (main)
$ mkdir praktikum-5

LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritma (main)
$ cd praktikum-5

LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritma/praktikum-5 (main)
$ npm iniy -y
Unknown command: "iniy"


Did you mean this?
  npm init # Create a package.json file
To see a list of supported npm commands, run:
  npm help

LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritma/praktikum-5 (main)
$ npm init -y
Wrote to D:\Praktikum Struktur Data dan Algoritma\2025573010066_strukturdataalgoritma\praktikum-5\package.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

8. Ketik touch 01-intro-kompleksitas.js di aplikasi git bash
9. Buka file nya di visual code, terus ketik program yang di perintahkan
10. Balik ke aplikasi gitbash dan ketik node 01-intro-kompleksitas.js

```
LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritma/praktikum-5 (main)
$ touch 01-intro-kompleksitas.js

LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritma/praktikum-5 (main)
$ code .

LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritma/praktikum-5 (main)
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 1 ms
O(n^2) cariPasangan : 14 ms

Hasil sama? true
```

11. Ketik lagi touch 02-desctructuring-spread.js di gitbash
12. Buka Kembali di visual code, terus ketik program yang di perintahkan
13. Balik ke aplikasi gitbash dan ketik node 02-desctructuring-spread.js

```
LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritma/praktikum-5 (main)
$ touch 02-destructuring-spread.js

LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritm
a/praktikum-5 (main)
$ node 02-destructuring-spread.js
=== O(1) – selalu cepat ===
10
true

=== O(log n) – Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) – Linear Search ===
Max dari 1000 elemen: 997

=== O(n^2) – Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2^n) – Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2^n):
68 ms
Waktu fib(35) O(n) memoization:
0 ms
```

14. Ketik lagi touch 03-space-complexity.js di gitbash
15. Buka Kembali di visual code, terus ketik program yang di perintahkan
16. Balik ke aplikasi gitbash dan ketik node 03-space-complexity.js

```
LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritm
a/praktikum-5 (main)
$ touch 03-space-complexity.js

LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritm
a/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : undefined
Elemen unik: 7
```

17. Ketik lagi touch 04-refactor.js di gitbash
18. Buka Kembali di visual code, terus ketik program yang di perintahkan
19. Balik ke aplikasi gitbash dan ketik node 04-refactor.js

```
LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritm
a/praktikum-5 (main)
$ touch 04-refactor.js

LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritm
a/praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n^2): 0 ms
Duplikat CEPAT O(n) : 1 ms

Frekuensi: { a: 2, b: 2, c: 1, d: 1 }
```

3.3 Hasil Praktikum

1. **Output tugas 1:**

```
C:\Program Files\nodejs\node.exe .\tugas\tugas-1.js
=== INTERSECTION ===
> Lambat: (2) [2, 3]
> Cepat : (2) [2, 3]

=== ANAGRAM ===
[eat, tea, ate] [tan, nat] [bat]

=== KUADRAT ===
Lambat: true
Cepat : false

=== BENCHMARK ===
Intersection Lambat: 31 ms
Intersection Cepat : 1 ms
Kuadrat Lambat: 2 ms
Kuadrat Cepat : 0 ms
```

2. **Output tugas 2**

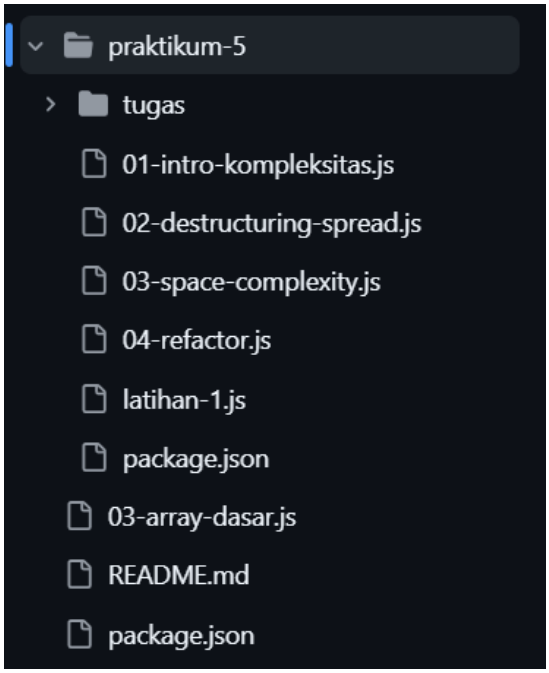
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL
C:\Program Files\nodejs\node.exe .\tugas\tugas-2.js

=== n = 100 ===
O(1)    : 0 ms
O(n)    : 0 ms
O(nlogn): 0 ms
O(n^2)  : 0 ms

=== n = 500 ===
O(1)    : 0 ms
O(n)    : 0 ms
O(nlogn): 0 ms
O(n^2)  : 2 ms

=== n = 1000 ===
O(1)    : 0 ms
O(n)    : 0 ms
O(nlogn): 0 ms
O(n^2)  : 2 ms

=== n = 5000 ===
O(1)    : 0 ms
O(n)    : 0 ms
O(nlogn): 0 ms
O(n^2)  : 23 ms

=== n = 10000 ===
O(1)    : 0 ms
O(n)    : 0 ms
O(nlogn): 0 ms
O(n^2)  : 80 ms
```

3.4 Submit ke GitHub

1. Langkah pertama masuk akun Git Bash
2. Ketik cd /d
3. Ketik lagi untuk masuk ke folder cd "/d/Praktikum Struktur Data dan Algoritma"
4. Ketik untuk masuk ke repository cd 2025573010066_strukturdataalgoritma
5. Kemudian masuk ke folder praktikum 5 yaitu cd praktikum-5
6. Kembali ke repository cd ..
7. Git add .
8. Git commit-m"praktikum 5: Analisis Kompleksitas Algoritma"
9. Git push origin main.
10. Tunggu sebentar untuk proses summit

```
LOQ@LAPTOP-337SK54S MINGW64 /d/Praktikum Struktur Data dan Algoritma/2025573010066_strukturdataalgoritm
a (main)
$ git push origin main
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 12 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (18/18), 5.80 KiB | 456.00 KiB/s, done.
Total 18 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/ZahwaAqiilah01/2025573010066_strukturdataalgoritma.git
   4f44493..b9187d5  main -> main
```

11. Buka github dan buka bagian repository jika benar tampilannya seperti ini, buka github dan buka bagian repository



BAB IV
PENUTUP

1.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa analisis kompleksitas algoritma sangat penting dalam menentukan efisiensi suatu program. Melalui konsep Big O Notation, dapat diketahui bagaimana waktu eksekusi dan penggunaan memori meningkat seiring bertambahnya ukuran data.

Hasil percobaan menunjukkan bahwa algoritma dengan kompleksitas rendah seperti O(1) dan O(n) memiliki performa yang lebih cepat dibandingkan algoritma dengan kompleksitas tinggi seperti $O(n^2)$ dan $O(n^3)$. Selain itu, penggunaan struktur data yang tepat seperti Set atau Object dapat membantu mengoptimalkan algoritma sehingga menjadi lebih efisien.

Dengan demikian, pemahaman terhadap analisis kompleksitas algoritma membantu dalam menulis kode yang lebih optimal, cepat, dan hemat memori, serta menjadi dasar penting dalam pengembangan program yang lebih kompleks di masa mendatang.

1.2 Saran

Mahasiswa disarankan untuk lebih sering berlatih dalam menganalisis kompleksitas algoritma agar semakin memahami perbedaan efisiensi setiap metode yang digunakan. Selain itu, dalam penulisan program sebaiknya selalu mempertimbangkan pemilihan algoritma dan struktur data yang tepat agar kinerja program menjadi lebih optimal, terutama saat menangani data dalam jumlah besar.

Selain itu, disarankan untuk melakukan pengujian program dengan berbagai variasi ukuran data agar dapat melihat perbedaan performa secara lebih jelas. Mahasiswa juga perlu memperdalam materi lanjutan terkait struktur data dan optimasi algoritma, serta memanfaatkan tools seperti Node.js dan Visual Studio Code secara maksimal untuk mendukung proses pembelajaran dan pengembangan program.